

QT&C++ project 보고서, 김도훈, 2026406041

<목차>

1. 프로젝트 개요
2. 게임 제작 동기
3. 게임 진행 방식
4. QT UI 구성
5. 게임 화면 출력 방법
6. 플레이어 키 입력 처리
7. 플레이어 이동과 ROS2 연결
8. Qt Signal을 이용한 좀비 화면 갱신
9. 게임 상태 UI 출력
10. GAME START / GAME OVER / GAME CLEAR UI
11. QTimer를 이용한 게임 시간 처리
12. 시야 시스템 구현
13. 카메라 시스템
14. 깨달음 점

1. 프로젝트 개요

본 프로젝트는 ROS2와 Qt를 이용하여 제작한 2D 좀비 생존 게임이다. Qt는 키보드 입력, 게임 화면 출력, 아이템 처리와 같은 게임 진행을 담당하고 ROS2는 플레이어와 좀비의 위치 데이터 전달 및 좀비 생성 기능을 담당한다. 플레이어는 맵을 탐색하여 식량 3개를 획득한 뒤 빨간 병커에 도착해야 한다. 게임이 진행될수록 좀비의 수가 증가하고 시야가 감소하며 플레이어는 고양이, 통조림, 햇불 등의 요소를 이용하여 생존해야 한다. 좀비의 이동에는 A* 알고리즘을 사용하였으며 플레이어와 좀비의 위치 정보는 ROS2 Topic을 통해 전달하였다. 좀비 생성과 같이 한 번의 동작을 요청하는 기능에는 ROS2 Service를 사용하였다.

2. 게임 제작 동기

처음에는 좀비를 피해 미로를 탈출하는 게임을 제작하려고 하였다. 하지만 미로 탈출 방식은 게임 구조가 단순하고 다양한 기능을 적용하기 어렵다고 판단하였다. 따라서 자유롭게 맵을 이동할 수 있는 좀비 생존 게임으로 방향을 변경하였다. 식량 3개를 획득한 뒤 병커에 복귀하는 목표를 추가하였고 시간이 지날수록 좀비가 증가하도록 하였다. 또한 플레이어만 좀비를 피하는 일반적인 구조에서 벗어나기 위해 좀비가 고양이를 피하도록 하였으며 게임 후반의 난이도를 높이기 위해 시야 감소와 햇불을 이용한 시야 회복 기능도 추가하였다.

3. 게임 진행 방식

게임이 시작되면 플레이어는 WASD 키를 이용하여 맵을 탐색한다. 초록색 건물을 조사하여 식량을 획득하고 총 3개의 식량을 모은 뒤 빨간 병커에 도착하면 게임이 클리어된다. 게임 진행 중에는 일정 시간마다 좀비가 생성되고 플레이어의 시야가 감소한다. 좀비와 충돌하면

GAME OVER가 된다. 플레이어는 횃불을 이용하여 시야를 회복할 수 있으며 통조림을 사용하여 고양이가 일정 시간 플레이어를 따라오도록 할 수 있다. 고양이 주변에는 좀비가 접근하지 않도록 하여 생존에 활용할 수 있도록 하였다. 게임 진행 중 좀비와 충돌하면 GAME OVER가 된다.

4. Qt UI 구성

본 프로젝트의 게임 화면은 Qt Designer를 이용하여 구성하였다. Qt Designer에서 게임 화면을 출력하기 위한 QLabel과 현재 게임 정보를 보여주기 위한 여러 개의 Label을 배치하였다.

label_map	실제 게임 화면 출력
label_food	획득한 식량 개수 표시
label_zombie_count	현재 존재하는 좀비 수 표시
label_game_status	통조림 개수, 시야 등의 게임 상태 표시
label_game_message	GAME START, GAME OVER, GAME CLEAR 표시

게임의 실제 맵은 label_map 위에 출력된다.

5. 게임 화면 출력 방법

게임 화면 출력은 GameRenderer 클래스에서 담당하도록 하였다. MainWindow에서 게임 화면을 갱신해야 할 때 updateGameScreen() 함수를 실행한다. 구조는 다음과 같은 방식이다.

```
void MainWindow::updateGameScreen()
{
    QPixmap gamePixmap = gameRenderer_.renderMap(
        &mapManager_,
        &buildingManager_,
        &player_,
        zombies_,
        cats_,
        cannedFoods_,
        torches_,
        visionManager_.getVisionRadius()
    );
    ui->label_map->setPixmap(gamePixmap);
}
```

renderMap()에는 현재 게임 상태를 전달한다. 플레이어 위치, 좀비 위치, 고양이 위치, 아이템 상태, 건물 정보, 현재 시야 범위 등을 전달하면 GameRenderer에서 하나의 QPixmap에 모든 오브젝트를 그린다.

6. 플레이어 키 입력 처리

플레이어 이동은 MainWindow에서 키보드 입력을 직접 처리하도록 구현하였다. Qt에서는 keyPressEvent()를 이용하여 키보드 입력을 받을 수 있다.

```
void MainWindow::keyPressEvent(QKeyEvent *event)
{
    int nextRow = player_.getRow();
    int nextCol = player_.getCol();
    if (event->key() == Qt::Key_W)
        nextRow--;
    else if (event->key() == Qt::Key_S)
        nextRow++;
    else if (event->key() == Qt::Key_A)
        nextCol--;
    else if (event->key() == Qt::Key_D)
        nextCol++;
    player_.moveTo(
        nextRow,
        nextCol,
        &mapManager_
    );
}
```

W, A, S, D 입력에 따라 이동할 row와 col을 계산하고 Player 클래스의 이동 함수를 호출한다. 하지만 키를 눌렀다고 무조건 이동하는 것은 아니다. Player 클래스에서는 MapManager가 가지고 있는 Collision Map을 확인하여 해당 위치로 이동할 수 있는지 확인한다. 따라서 플레이어가 건물을 통과하지 않도록 구현하였다.

7. 플레이어 이동과 ROS2 연결

플레이어가 이동한 뒤에는 새로운 위치를 좀비 AI 노드에 알려주어야 한다. 따라서 이동 후 QNode의 Publisher를 호출한다.

```
qnode->publishPlayerPosition(
    player_.getRow(),
    player_.getCol()
);
```

QNode에서는 해당 위치를 ROS2 Topic으로 Publish한다.

```
void QNode::publishPlayerPosition(int row, int col)
{
    auto msg = std_msgs::msg::Int32MultiArray();
    msg.data.push_back(row);
    msg.data.push_back(col);
    playerPositionPublisher_->publish(msg);
}
```

```
}
```

ZombieAiNode는 Topic을 Subscribe하여 플레이어의 현재 위치를 A* 알고리즘의 목표 위치로 사용한다.

8. Qt Signal을 이용한 좀비 화면 갱신

좀비 이동 결과는 ZombieAiNode에서 /zombie_positions Topic으로 Publish한다. QNode는 해당 Topic을 Subscribe한다. 하지만 ROS2 Subscriber의 Callback에서 직접 Qt 화면을 변경하지 않고 Qt Signal을 이용하여 MainWindow로 데이터를 전달하도록 하였다. 예를 들어 다음과 같은 Signal을 사용한다.

signals:

```
void zombiePositionsReceived(QVector<int> positions);
```

좀비 위치 Topic을 수신하면 Signal을 발생시킨다.

```
emit zombiePositionsReceived(zombieData);
```

MainWindow에서는 이 Signal을 자신의 함수와 연결한다.

```
connect(qnode,&QNode::zombiePositionsReceived,this,&MainWindow::updateZombiePositions)
```

이를 통해 ROS2 통신 결과가 실제 Qt 게임 화면의 좀비 움직임으로 연결된다.

9. 게임 상태 UI 출력

게임 화면뿐만 아니라 플레이어가 현재 상태를 확인할 수 있도록 UI에 여러 정보를 표시하였다. 예를 들어 식량 개수는 다음과 같은 방식으로 표시할 수 있다.

```
void MainWindow::updateGameInfo()
{
    ui->label_food->setText(
        QString("FOOD : %1 / 3")
        .arg(player_.getFoodCount())
    );
}
```

좀비 수도 현재 zombies_의 크기를 이용하여 출력한다.

```
ui->label_zombie_count->setText( QString("ZOMBIE : %1") .arg(zombies_.size()));
```

통조림과 시야 정보도 하나의 Label을 이용하여 출력할 수 있다.

```

ui->label_game_status->setText(
    QString("CAN : %1    VISION : %2")
    .arg(cannedFoodCount_)
    .arg(visionManager_.getVisionRadius())
);

```

따라서 플레이어는 게임을 진행하면서 현재 가지고 있는 식량, 통조림, 시야 상태, 좀비 수를 실시간으로 확인할 수 있다.

10. GAME START / GAME OVER / GAME CLEAR UI

게임의 시작과 종료 상태를 확실하게 보여주기 위해 label_game_message를 별도로 만들었다. 평소에는 해당 Label을 숨겨둔다.

```
ui->label_game_message->hide();
```

게임 시작 시에는 다음과 같이 표시한다. showGameMessage("GAME START", "white");

게임 오버가 발생하면: showGameMessage("GAME OVER", "red");

게임 클리어가 발생하면: showGameMessage("GAME CLEAR", "lime"); 를 출력하도록 하였다. 함수는 다음과 같은 형태로 구성할 수 있다.

```

void MainWindow::showGameMessage(
    QString text,
    QString color
){
    ui->label_game_message->setText(text);
    ui->label_game_message->setStyleSheet(
        "font-size: 100px;"
        "font-weight: bold;"
        "color: " + color + ";"
        "background-color: rgba(0, 0, 0, 150);"
    );
    ui->label_game_message->show();
    ui->label_game_message->raise();
}

```

raise()를 사용하면 label_game_message가 게임 화면보다 위에 출력되기 때문에 게임 종료 문구를 크게 보여줄 수 있다.

11. QTimer를 이용한 게임 시간 처리

게임에는 시간에 따라 동작해야 하는 기능이 여러 개 존재한다.

이를 하나의 반복문에서 모두 처리하지 않고 Qt의 QTimer를 기능별로 사용하였다.

예를 들어 시야 감소 Timer는 다음과 같이 설정하였다.

```
darknessTimer_ = new QTimer(this);
```

```
connect(  
    darknessTimer_,  
    &QTimer::timeout,  
    this,  
    &MainWindow::decreaseVision  
);
```

```
darknessTimer_->start(15000);
```

위 코드에서는 15초마다 decreaseVision()이 실행된다.

```
void MainWindow::decreaseVision()  
{  
    visionManager_.decreaseVision();  
    updateGameScreen();  
}
```

좀비 생성에도 별도의 Timer를 사용하였다.

```
zombieSpawnRequestTimer_ = new QTimer(this);
```

```
connect(  
    zombieSpawnRequestTimer_,  
    &QTimer::timeout,  
    this,  
    &MainWindow::requestZombieSpawn  
);
```

이처럼 각 기능마다 Timer를 분리하면 게임 진행 시간을 독립적으로 관리할 수 있다.

12. 시야 시스템 구현

시야 범위는 VisionManager 클래스에서 관리하였다. 게임이 시작될 때 일정한 시야 범위를 가지고 있으며 시간이 지나면 감소한다.

```
void VisionManager::decreaseVision()  
{  
    if (visionRadius_ > minimumVisionRadius_)  
        visionRadius_--;  
}
```

햇불을 획득하면 시야 범위를 증가시킨다.

```
void VisionManager::increaseVision(int amount)  
{  
    visionRadius_ += amount;  
    if (visionRadius_ > torchMaximumVisionRadius_)  
        visionRadius_ = torchMaximumVisionRadius_;
```

```
}
```

실제 어둠 표현은 GameRenderer에서 처리한다. 플레이어의 위치를 중심으로 일정 거리 안 쪽은 밝게 표시하고 그보다 멀리 있는 위치에는 검은색 투명 레이어를 출력한다. 게임 시간이 지날수록 이 밝은 영역이 감소하기 때문에 플레이어가 주변 상황을 확인하기 어려워진다. 이를 통해 생존 게임의 긴장감을 높였다.

13. 카메라 시스템

처음에는 60 * 78 크기의 전체 게임 맵을 한 화면에 출력하였다. 하지만 전체 맵이 보이면 멀리 있는 좀비, 건물, 횃불 등의 위치를 플레이어가 쉽게 파악할 수 있기 때문에 생존 게임의 긴장감이 감소하였다. 이를 해결하기 위해 전체 맵을 렌더링한 후 플레이어 주변 일정 부분만 잘라서 화면에 표시하는 카메라 시스템을 추가하였다.

카메라는 약 39 * 30 Tile 영역을 사용하였다.

```
int cameraCols = 39;
int cameraRows = 30;
int cameraWidth = cameraCols * 16;
int cameraHeight = cameraRows * 16;
```

플레이어 위치를 기준으로 카메라 시작 위치를 계산한다.

```
int cameraX = player_.getCol() * 16 - cameraWidth / 2;
int cameraY = player_.getRow() * 16 - cameraHeight / 2;
그리고 전체 게임 화면에서 해당 부분만 복사한다.
```

```
QPixmap cameraPixmap =
    gamePixmap.copy(
        cameraX,
        cameraY,
        cameraWidth,
        cameraHeight
    );
```

마지막으로 label_map 크기에 맞게 확대한다.

```
ui->label_map->setPixmap(
    cameraPixmap.scaled(
        ui->label_map->size(),
        Qt::IgnoreAspectRatio,
        Qt::FastTransformation
    )
);
```

이를 통해 실제 게임 맵은 넓게 유지하면서 플레이 중에는 플레이어 주변만 확대하여 확인할 수 있도록 하였다.

14. 깨달음 점

이번 Qt&C++ 프로젝트에서 ui를 잘 구성하고 싶었지만 시간과 실력의 이유로 내가 원하는 만큼의 ui를 구성하지 못했다. 저번 A*를 활용한 미로 과제를 할 때 A* 알고리즘을 활용하여 나만의 프로젝트를 만들어보고 싶었는데 생각을 했었고 그걸 이 프로젝트에서 할 수 있어서 프로젝트를 하면서 힘들지 않았다. 하지만 A* 알고리즘을 0부터 나의 손으로 개발을 하는 건 불가능했다. C++에 익숙하지 않은 건지 A* 알고리즘에 대해 잘못 이해하고 있는 건지 더 알아봐야 할 거 같다. 또한 topic과 service 같은 ROS2 통신 관련 개념들은 여전히 어려웠다. 이것을 Qt와 C++과 연결을 시키는 건 더 어려웠다. 시간이 지나면 이것을 나 스스로작성 할 줄 알아야할 텐데 어떻게 공부해야할지 막막한 생각이 들었다. 현재 나는 LLM 없이는 코드를 완성하지 못한다. LLM이 아무리 발전한다고 하더라도 개인적으로는 코드는 손수 작성할 줄 알아야 한다고 생각한다. 앞으로 많이 사용될 C++, ROS2는 교육이 끝났을지라도 계속해서 공부해나갈 것이다.